



AI Middleware and RAG Pipelines

Task

[Visit our website](#)

[Quick start](#)

[AI Middleware and RAG Pipelines](#)

Introduction

Over the previous lessons, you have developed a solid understanding of the technologies that power modern Artificial Intelligence (AI) systems. You began by exploring the principles of AI and machine learning before learning how embeddings allow computers to represent the meaning of information mathematically. You then examined how transformer architectures and large language models (LLMs) generate human-like responses, followed by practical applications of generative AI in web development. Most recently, you learned how prompt engineering enables developers to communicate effectively with AI systems and produce more reliable outputs.

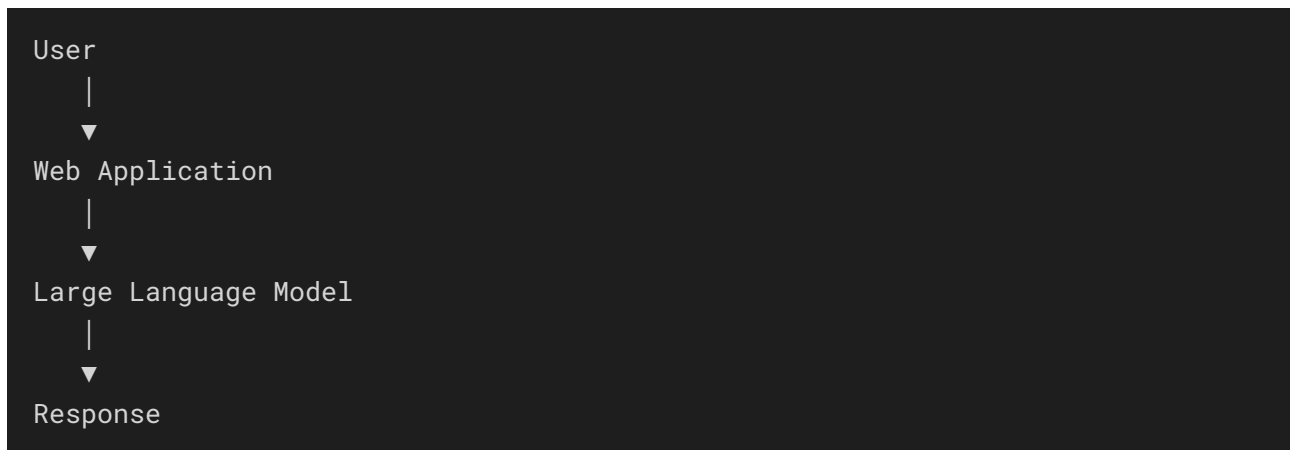
Although these topics explain how AI models work and how developers interact with them, they do not fully explain how modern AI-powered applications are built. In practice, successful AI applications involve far more than sending a prompt to a language model and displaying the response. Enterprise AI systems often retrieve information from company databases, search internal documentation, process user requests, apply business rules, and combine information from multiple services before generating an answer. This orchestration layer is commonly referred to as AI middleware.

One of the most important architectural patterns used within AI middleware is **Retrieval-Augmented Generation (RAG)**. Rather than relying solely on the information contained within a language model, a RAG system retrieves relevant information from an external knowledge source and incorporates that information into the prompt before asking the model to generate a response. This allows AI systems to answer questions using current, domain-specific, or private information that may not exist within the model's original training data.

In this lesson, you will learn how AI middleware coordinates intelligent workflows and how Retrieval-Augmented Generation improves the accuracy and usefulness of AI applications. You will also explore the components that make up a RAG pipeline and discover how frameworks such as `LangChain.js` help developers build these systems. By the end of the lesson, you will understand the architecture behind many of today's AI-powered applications and be able to implement a simplified retrieval pipeline without relying on commercial AI services.

From Simple AI Applications to Intelligent Systems

When many developers first begin experimenting with AI, they build applications that follow a very simple workflow. A user enters a question, the application sends that question directly to a language model, and the generated response is displayed on the screen. The workflow is straightforward:



This architecture works well for general-purpose questions such as:

`Explain JavaScript promises.`

or

`Write a CSS animation for a button.`

Because these questions involve common programming knowledge, the language model can usually produce useful responses without requiring any additional information. However, problems begin to appear when applications require knowledge that is specific to a particular organisation, product, or user. Imagine that a university develops an AI assistant capable of answering questions about its courses. A student asks:

`What are the entry requirements for the Web Development Bootcamp?`

A general-purpose language model cannot reliably answer this question because the information may not have been available during training, or it may have changed since then. Even worse, the model might generate an answer that sounds convincing but is completely incorrect. This limitation is one of the biggest challenges when building practical AI applications.

Why Standalone Large Language Models Have Limitations

Large language models are trained on enormous collections of publicly available text. During training, they learn statistical relationships between words, sentences, and concepts. This allows them to answer questions, generate code, summarise documents, and perform many other language-related tasks. Despite their impressive capabilities, language models have important limitations. One limitation is that they do not automatically know about information that exists outside their training data. They cannot access your company's internal documentation, private databases, customer records, or application-specific knowledge unless that information is provided during the conversation.

Another limitation is that training a language model is an extremely expensive process. Once a model has been trained, its knowledge does not automatically update every time new information becomes available. If an organisation changes its pricing, publishes new documentation, or updates its policies, the language model does not immediately become aware of those changes. Developers could theoretically retrain or fine-tune a model whenever new information becomes available. However, retraining large language models requires enormous computational resources, specialised expertise, and significant financial investment. For most organisations, this approach is neither practical nor necessary. Instead, developers use retrieval techniques to provide current information whenever a user submits a request.

What Is AI Middleware?

Before exploring Retrieval-Augmented Generation, it is useful to understand the concept of middleware. In traditional software engineering, middleware is software that sits between different systems and allows them to communicate. It acts as an intermediary that coordinates requests, processes information, and connects multiple services into a single workflow. Consider a typical e-commerce application. When a customer places an order, the request is rarely handled by a single system. Instead, multiple services work together. One service verifies product availability, another processes payments, another updates inventory, while another arranges shipping.

The middleware coordinates these interactions so that the user experiences a single, seamless process. AI applications work in a similar way. Rather than communicating directly with a language model, many modern AI applications first perform several intermediate tasks. They may search documents, retrieve customer information, apply security rules, filter sensitive data, or combine information from multiple sources before generating a response. The software responsible for coordinating these activities is known as **AI middleware**. Rather than viewing an AI model as the centre of the

application, it is more accurate to think of the language model as one component within a much larger intelligent workflow.

The Role of AI Middleware in Modern Applications

As AI systems become more sophisticated, middleware has become increasingly important because it provides capabilities that language models cannot perform independently. For example, imagine a company developing an AI-powered customer support assistant. When a customer asks:

Where is my order?

The AI should not simply guess the answer. Instead, the application might perform the following steps:

1. Verify the customer's identity.
2. Retrieve the customer's recent orders.
3. Query the shipping provider.
4. Collect the latest delivery information.
5. Construct a prompt containing the retrieved information.
6. Ask the language model to generate a natural-language response.
7. Return the response to the customer.

Although the user only sees a single answer, several independent systems have worked together behind the scenes. This coordinated workflow is made possible by AI middleware. Modern AI applications frequently integrate with:

- Document repositories
- Company knowledge bases
- Product catalogues
- Customer databases
- Search engines
- External APIs
- Internal business systems

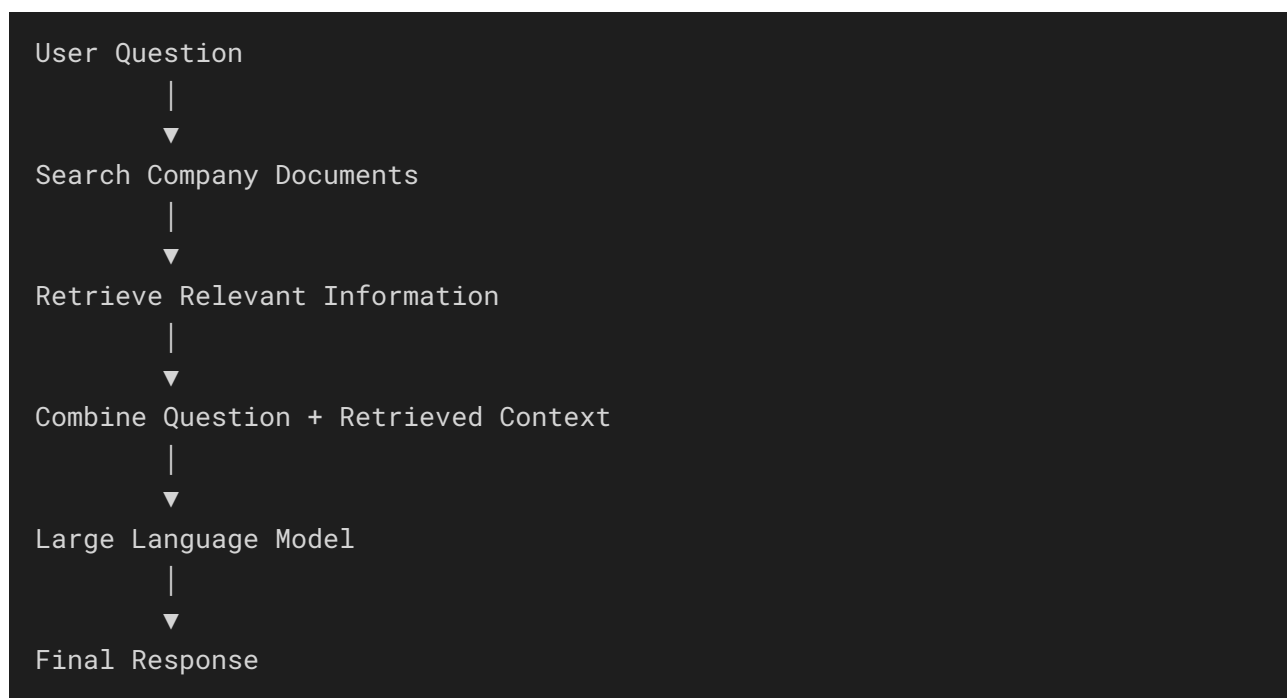
The middleware determines which information is needed, retrieves it, prepares it for the language model, and manages the overall workflow. Without middleware, AI applications would be limited to whatever information the language model already knows.

Understanding Retrieval-Augmented Generation (RAG)

One of the most widely adopted middleware patterns in AI development is **Retrieval-Augmented Generation**, commonly abbreviated as **RAG**. Retrieval-Augmented Generation is an architectural approach that combines two separate processes. The first process retrieves relevant information from an external knowledge source. The second process uses that retrieved information to generate a more accurate response. Rather than asking a language model to answer a question from memory alone, the application first gathers supporting information and then includes that information within the prompt. Imagine asking an AI assistant:

What is our company's refund policy?

Instead of relying entirely on the language model's existing knowledge, a RAG system would first search the company's policy documents, locate the relevant section, and include that information before requesting a response. The workflow becomes:



This additional retrieval step dramatically improves the accuracy and reliability of the generated answer.

Why Retrieval-Augmented Generation Exists

Retrieval-Augmented Generation was developed to solve several practical challenges associated with large language models. The first challenge is knowledge freshness. Business information changes constantly. New products are released, policies are updated, documentation evolves, and customer information changes every day. Retraining a language model every time information changes would be impractical.

The second challenge is access to private information. Most organisations possess large collections of internal knowledge that should never become part of a publicly trained language model. This may include confidential documents, engineering specifications, employee manuals, or customer records. RAG allows AI systems to access this information securely at the time of the request without permanently storing it inside the language model.

The third challenge is reducing hallucinations. As you learned in the previous lesson, language models sometimes generate convincing but incorrect information. By supplying relevant source material alongside the user's question, developers reduce the amount of guessing the model needs to perform. Instead of asking the model to invent an answer, the application encourages it to answer using retrieved evidence.

RAG Versus Fine-Tuning

Students often confuse Retrieval-Augmented Generation with fine-tuning because both approaches improve AI systems. Although their goals are similar, they solve different problems. Fine-tuning modifies the behaviour of the language model itself. During fine-tuning, the model undergoes additional training using specialised datasets so that it becomes better at performing certain tasks or adopts a particular writing style. Retrieval-Augmented Generation does not change the model at all.

Instead, RAG improves responses by providing additional information during inference, which is the stage when the model generates a response for the user. An easy way to think about the difference is through a classroom analogy. Imagine a student writing an examination. Fine-tuning is similar to giving the student additional months of study before the examination begins. Their knowledge has permanently increased because they have learned new material.

Retrieval-Augmented Generation is more like allowing the student to consult a reference book during the examination. The student's knowledge has not changed, but they can retrieve accurate information whenever it is needed. Because information can be updated independently of the language model, RAG is generally much easier to maintain than repeatedly fine-tuning a model whenever business information changes.

High-Level Architecture of a RAG System

Although RAG systems can become extremely sophisticated, they all follow the same fundamental architecture. The process begins when a user submits a question through a web application or chatbot. The application does not immediately send the question to a language model. Instead, the middleware first searches one or more knowledge sources for information related to the user's request. These knowledge sources may include documents, databases, manuals, product catalogues, frequently asked questions, or internal reports.

Once the most relevant information has been retrieved, the middleware combines that information with the user's original question to construct an enhanced prompt. Only then is the prompt sent to the language model. Finally, the generated response is returned to the user. This architecture allows the language model to generate responses based not only on its own training but also on information that exists within the organisation's knowledge base.

In the next part of this lesson, you will move beyond the high-level architecture and explore how knowledge sources are transformed into searchable information. You will learn how documents are loaded, divided into meaningful chunks, converted into vector representations, and retrieved using semantic similarity before becoming part of the final AI prompt.

Building a Retrieval Pipeline

In the previous section, you learned that Retrieval-Augmented Generation improves AI applications by retrieving relevant information before a language model generates a response. Rather than expecting the model to answer every question using only its training data, a RAG system searches external knowledge sources and provides the most relevant information as additional context. This raises an important question. How does a computer search thousands of documents and identify only the pieces of information that are relevant to a user's question?

The answer lies in a carefully designed retrieval pipeline. This pipeline prepares documents, converts them into searchable representations, identifies the most relevant information, and delivers that information to the language model. Each stage plays an important role in improving the quality and accuracy of AI-generated responses.

In this section, you will explore each component of the retrieval pipeline and learn how frameworks such as LangChain.js help developers connect these components into a complete AI workflow.

Knowledge Sources

Every Retrieval-Augmented Generation system begins with a knowledge source. This is the collection of information that the AI application can access when answering questions. Unlike a standalone language model, which depends entirely on its training data, a RAG application can search information that belongs to a specific organisation, application, or user.

Knowledge sources come in many different forms. An organisation may choose to retrieve information from PDF documents, web pages, product catalogues, Frequently Asked Questions (FAQs), databases, support tickets, markdown files, spreadsheets, or structured JSON files. Consider an online learning platform. The organisation may have thousands of pages describing courses, assessments, grading policies, deadlines, and student support services. Rather than embedding all of this information directly into a language model, the platform stores these resources separately and retrieves only the information required for each question.

Suppose a student asks:

```
What topics are covered in the React module?
```

The retrieval system searches the available learning resources, identifies the documents that discuss the React curriculum, and forwards the relevant information to the language model. This approach allows organisations to update their knowledge simply by modifying their documents rather than retraining an AI model.

Document Loaders

Knowledge sources often exist in many different formats. Before documents can be searched, they need to be loaded into the retrieval pipeline. A document loader is responsible for reading information from its original source and converting it into a format that the remainder of the pipeline can process. For example, a loader may read information from:

- A PDF document
- A Microsoft Word document
- A web page
- A JSON file
- A database
- A CSV file

Although these sources differ significantly, the retrieval pipeline treats them as collections of text that can later be divided into smaller sections.

Frameworks such as LangChain.js include document loaders for many common file formats. This allows developers to build retrieval systems without writing custom code for every type of document. Imagine building an AI assistant for a software company. The company's documentation exists as hundreds of markdown files stored in a Git repository. A document loader reads each file, extracts the text, and prepares it for the next stage of processing. Loading documents is therefore the first step in transforming raw information into searchable knowledge.

Why Documents Cannot Be Searched as a Whole

A common misconception is that an AI system searches complete documents when answering questions. In reality, searching entire documents is often ineffective. Imagine a technical manual containing one hundred pages of documentation. A user asks:

```
How do I reset my password?
```

Only a few paragraphs within the manual actually answer this question. If the retrieval system returns the entire document, the language model receives far more information than necessary. This wastes computational resources, increases processing time, and may even reduce the quality of the generated response by introducing unrelated information. Instead, modern retrieval systems divide documents into much smaller sections. This process is known as **chunking**.

Document Chunking

Chunking is the process of dividing large documents into smaller pieces called chunks. Each chunk represents a manageable section of information that can later be searched independently. Rather than searching an entire manual, the retrieval system searches hundreds or even thousands of smaller chunks. Imagine a programming handbook containing chapters on JavaScript, React, Node.js, databases, testing, and deployment. After chunking, the handbook may produce individual sections such as:

- Variables and Data Types
- Loops and Iteration
- React Components
- React Hooks
- Express Routing
- REST APIs
- Authentication
- Deployment

When a developer asks about React Hooks, the retrieval system searches these smaller chunks instead of scanning the complete handbook. This makes retrieval faster and considerably more accurate.

Choosing an Appropriate Chunk Size

Although chunking sounds straightforward, determining the correct chunk size requires careful consideration. Chunks that are too small may lose important context. Suppose the sentence:

```
The submit button validates the form before sending data to the server.
```

is separated from the surrounding explanation. Without additional context, the language model may struggle to understand how the validation process works. On the other hand, chunks that are too large introduce unnecessary information. If an entire chapter is stored as a single chunk, only a small percentage of that information may be relevant to the user's question. Finding the right balance is therefore an important part of building an effective retrieval system. Many production systems also allow chunks to overlap slightly. Overlapping preserves relationships between neighbouring sections and reduces the chance that important information is split across chunk boundaries.

Revisiting Embeddings

Earlier in this module, you learned that embeddings convert words, sentences, or documents into numerical vectors that capture semantic meaning. Retrieval-Augmented Generation builds directly upon this concept. Once documents have been divided into chunks, each chunk is converted into an embedding. Instead of storing text alone, the system stores both the original text and its mathematical representation. This numerical representation allows the retrieval system to compare meanings rather than simply matching keywords. For example, imagine that a knowledge base contains the following sentence:

```
Students should submit all assignments before the due date.
```

A user asks:

```
When must coursework be handed in?
```

Although the words "coursework" and "assignments" differ, their embeddings occupy similar positions within vector space. The retrieval system recognises this semantic relationship and retrieves the relevant chunk, even though the wording differs. This capability is one of the reasons RAG systems outperform traditional keyword search.

Vector Representations

Once embeddings have been generated, every document chunk becomes a point within a high-dimensional mathematical space. You can imagine this space as a map where similar ideas appear close together while unrelated ideas appear farther apart. Earlier in the course, you explored similarity measurements such as Euclidean distance and cosine similarity. These mathematical techniques are now applied to document retrieval. When a user submits a question, the question itself is converted into an embedding. The retrieval system then compares the question's vector with every stored document vector.

Chunks with the smallest semantic distance or highest similarity score are considered the most relevant. The system retrieves these chunks and forwards them to the next stage of the pipeline. Notice that the retrieval process no longer depends on matching individual words. Instead, it searches for similar meanings.

Similarity Search

Similarity search forms the heart of every Retrieval-Augmented Generation system. Traditional search engines often rely on keyword matching. If the search terms do not appear exactly as written, relevant documents may be overlooked. Similarity search takes a different approach. Instead of asking:

`Does this document contain these words?`

the system asks:

`Does this document discuss the same idea?`

This distinction allows RAG systems to retrieve information even when users phrase questions differently from the original documents. Imagine an organisation storing documentation that uses the phrase:

`employee onboarding`

A new employee searches for:

`new starter orientation`

Although none of the keywords match exactly, the embeddings represent closely related concepts. The retrieval system therefore identifies the appropriate documentation. Semantic search significantly improves the flexibility and usefulness of AI-powered applications.

Retrievers

The component responsible for performing similarity search is known as the retriever. A retriever receives the user's question, converts it into an embedding, compares it with the stored document embeddings, and returns the most relevant results. Think of the retriever as a librarian working in an enormous digital library. Rather than reading every book, the librarian quickly identifies the sections most likely to answer the visitor's question. The retriever performs the same task using mathematics rather than human judgement.

Different retrievers use different strategies for ranking information. Some prioritise similarity scores, while others combine semantic similarity with keyword matching, metadata, filtering rules, or business logic. Regardless of the specific implementation, the objective remains the same: identify the information that is most likely to help answer the user's question.

From Retrieval to Context

Retrieving information is only part of the overall process. Once the relevant document chunks have been identified, they must be organised into a form that the language model can understand. Imagine that a retriever identifies four separate document sections related to a user's question. Simply sending these sections individually to the language model would not provide sufficient context. Instead, the middleware combines the retrieved information with the user's original question to create a richer prompt.

This process transforms isolated pieces of information into meaningful context that guides the language model toward producing a more accurate response. In the next section of this lesson, you will explore how prompt augmentation combines retrieved information with user questions, how `LangChain.js` coordinates these individual components, and how developers build complete Retrieval-Augmented Generation pipelines using middleware frameworks.

The next section builds upon these concepts by showing how retrieved information is combined with user prompts and orchestrated through `LangChain.js` to create complete AI middleware workflows capable of supporting real-world web applications.

Prompt Augmentation

By this point in the retrieval pipeline, the system has successfully identified the document chunks that are most relevant to the user's question. However, the language model has not yet generated a response. There is still one important step remaining before the request can be sent to the model. The retrieved information must be

combined with the user's original question in a structured way. This process is known as **prompt augmentation**.

Prompt augmentation is the stage where additional context is inserted into the prompt before it is sent to the language model. Rather than asking the model to answer a question using only its own knowledge, the application supplies carefully selected information that should be used when generating the response. This is one of the defining characteristics of the Retrieval-Augmented Generation. The language model is no longer working in isolation. Instead, it is supported by relevant information retrieved from external knowledge sources.

Consider the following user question:

How do I submit my final assessment?

Without Retrieval-Augmented Generation, the prompt sent to the language model might simply be:

How do I submit my final assessment?

A RAG application would instead retrieve the relevant information from its knowledge base and construct a richer prompt. For example:

```
Use the following information to answer the student's question.
```

```
Context:
```

```
You are an AI assistant for an online learning platform.
```

```
Students must submit all final assessments through the LMS before 11:59 PM  
on the due date. Late submissions are only accepted if an approved extension  
has been granted.
```

```
Question:
```

```
How do I submit my final assessment?
```

Notice that the language model now receives both the user's question and the supporting information needed to answer it accurately. The retrieved context effectively becomes part of the prompt, guiding the model toward a more reliable response.

Why Prompt Augmentation Improves AI Responses

Prompt augmentation improves AI performance because it reduces uncertainty. Earlier in this course, you learned that large language models generate responses by predicting the most appropriate sequence of words based on the information available

to them. When little context is provided, the model must make assumptions about what the user is asking. Additional context reduces the number of assumptions the model must make.

Imagine asking a friend for directions to a building. If you simply ask:

`Where is the library?`

your friend may not know which library you mean.

If instead you ask:

`Where is the engineering library on the university campus?`

the additional context makes the question much easier to answer. Prompt augmentation works in a similar way. It narrows the range of possible interpretations and provides evidence that the language model can use when constructing its response. This leads to responses that are generally more accurate, more relevant, and more consistent.

Constructing Effective Augmented Prompts

Although the concept of prompt augmentation is straightforward, constructing an effective augmented prompt requires careful design. A poorly structured prompt can overwhelm the language model with unnecessary information, while a prompt containing too little context may not provide enough evidence to answer the question accurately. Most RAG systems organise augmented prompts into three logical sections. The first section establishes the role or behaviour expected of the language model.

For example:

`You are a customer support assistant.`

or

`You are an AI tutor helping beginner web development students.`

The second section provides the retrieved context. This section contains the information identified during the retrieval stage. Only the most relevant document chunks should be included. The final section presents the user's original question. Organising prompts in this way helps the language model distinguish between instructions, supporting evidence, and the actual request. As you learned in the previous lesson on prompt engineering, well-structured prompts generally produce more consistent outputs than loosely organised instructions.

Managing Context Windows

Although language models are capable of processing large amounts of text, every model has a limit to how much information it can consider at one time. This limit is commonly referred to as the **context window**. The context window represents the maximum amount of text that can be processed during a single interaction. It includes everything supplied to the model, including system instructions, retrieved documents, conversation history, and the user's question.

Imagine attempting to read an entire textbook before answering a single question. Even for a human, this would be inefficient. Language models experience a similar challenge. Supplying excessive amounts of context increases computational cost and may reduce response quality if irrelevant information distracts the model from the main topic. For this reason, retrieval systems attempt to retrieve only the most relevant document chunks rather than entire documents. Good retrieval is therefore just as important as good prompting.

Ranking Retrieved Information

Not every retrieved document contributes equally to answering a question. Suppose a knowledge base contains five relevant document chunks.

1. One chunk explains the exact procedure the user is asking about.
2. Another briefly mentions the topic.
3. A third discusses a related concept without directly answering the question.
4. Rather than including every retrieved document, many retrieval systems rank the results according to relevance.
5. The highest-ranked chunks are placed at the beginning of the prompt because they are most likely to influence the generated response.

This ranking process improves both efficiency and accuracy. Some advanced retrieval systems also remove duplicate information or combine related document sections before constructing the final prompt. These optimisations help maximise the value of the available context window.

Introducing LangChain.js

As Retrieval-Augmented Generation pipelines become more sophisticated, managing every stage manually becomes increasingly difficult. Developers would need to write code to:

- Load documents.
- Split documents into chunks.
- Generate embeddings.
- Retrieve relevant information.
- Construct prompts.
- Send prompts to a language model.
- Process responses.
- Handle errors.

Writing all of this functionality from scratch would require a considerable amount of development effort. This is where **LangChain.js** becomes valuable. **LangChain.js** is an open-source JavaScript framework designed to help developers build applications powered by large language models. Rather than replacing AI models, **LangChain.js** provides the infrastructure needed to connect models with documents, databases, search systems, APIs, and other application components.

You can think of **LangChain.js** as the framework responsible for coordinating the entire AI workflow. Instead of writing every connection manually, developers assemble reusable building blocks that work together as a pipeline. This makes AI applications easier to build, easier to understand, and easier to maintain.

Understanding Chains

The name **LangChain** comes from one of its central ideas: **chains**. A chain represents a sequence of connected operations where the output from one step becomes the input for the next. This idea should already feel familiar. Throughout this lesson, you have seen that a Retrieval-Augmented Generation system follows a sequence of processing stages.

- The user submits a question.
- The system retrieves relevant documents.
- The retrieved information is combined with the original question.
- The language model generates a response.
- Each stage depends on the results of the previous stage.
- A chain formalises this workflow by linking each operation together.

Instead of viewing the application as a collection of unrelated functions, developers think of it as a pipeline through which information flows from one stage to the next and this approach produces applications that are easier to extend and debug.

Runnables

Modern versions of **LangChain.js** introduce another important concept known as **runnables**. A runnable represents any operation that can receive an input, perform a task, and return an output. For example, a runnable might:

- Load documents.
- Retrieve relevant information.
- Build a prompt.
- Generate a response.
- Format the output.

The advantage of runnables is that they can be combined to create larger workflows. Instead of writing one large function containing every step of the retrieval pipeline, developers build smaller reusable components that each perform a single responsibility. This design closely follows the software engineering principle of modularity. Smaller components are generally easier to test, reuse, and maintain than large monolithic functions.

Orchestrating the Retrieval Pipeline

One of the primary responsibilities of AI middleware is orchestration. Orchestration refers to coordinating multiple independent components so that they work together as a single system. Imagine an orchestra performing a concert. Each musician plays a different instrument, yet the conductor ensures that everyone performs at the correct time and in the correct order.

AI middleware performs a similar role which is coordinating the document loading, retrieval, prompt construction, language model interaction, and response generation into a single workflow that appears seamless to the user. Although each component performs an individual task, the user experiences a single interaction. This orchestration is one of the defining characteristics of modern AI-powered applications.

Bringing the Pipeline Together

You can now see how every component introduced throughout this lesson contributes to the overall Retrieval-Augmented Generation process.

- A user submits a question through a web application.
- The middleware converts the question into an embedding and searches the available knowledge base.
- The retriever identifies the most relevant document chunks.
- These chunks are inserted into an augmented prompt alongside the user's original question.
- The completed prompt is passed to the language model.
- Finally, the generated response is returned to the application.

Although each stage performs a relatively simple task, combining them creates an intelligent system capable of answering questions using information that exists outside the language model itself. This modular architecture is one of the main reasons why

Retrieval-Augmented Generation has become the preferred approach for building enterprise AI applications.

Conclusion

Prompt augmentation transforms retrieved information into meaningful context that language models can use when generating responses. Carefully constructed prompts, combined with relevant supporting information, allow AI systems to produce answers that are more accurate, current, and closely aligned with an organisation's knowledge.

Frameworks such as `LangChain.js` simplify this process by providing reusable components for loading documents, retrieving information, constructing prompts, and orchestrating complete AI workflows. Concepts such as chains and runnables encourage developers to think of AI applications as modular pipelines rather than isolated function calls.



Extra resource

Before starting this practical task, you should first work through the **Example RAG Pipeline** project included in your task files.

This example has been designed as a guided learning resource to reinforce the concepts introduced throughout this lesson. Unlike the practical task, which requires you to design and build your own solution, the example project demonstrates how each stage of a simplified Retrieval-Augmented Generation (RAG) pipeline works in a complete, runnable application.

As you work through the example, pay particular attention to how the application:

- Loads information from a local knowledge base.
- Divides documents into smaller searchable chunks.
- Retrieves the most relevant information for a user's question.
- Constructs an augmented prompt by combining retrieved information with the user's question.
- Generates a simulated AI response using the retrieved context.
- Coordinates the complete workflow through an AI middleware pipeline.

Each source file has been extensively documented with comments explaining not only what the code is doing, but also why that step is necessary within a Retrieval-Augmented Generation (RAG) system. Read the comments carefully and run

the example locally so that you can observe how information flows through the pipeline from the user's question to the final response.

Once you understand how the example project works, you should complete the practical task by developing **your own implementation**. Although the example demonstrates the overall architecture and programming techniques used in a RAG pipeline, your submission should be your own work and should follow the requirements outlined in the practical task below.



Practical task

Modern AI applications rarely rely on a large language model alone. Instead, they retrieve information from external knowledge sources before generating responses. This approach, known as Retrieval-Augmented Generation (RAG), enables AI applications to produce more accurate, relevant, and up-to-date answers.

In this practical task, you will build a simplified Retrieval-Augmented Generation pipeline for a web application. Rather than using a commercial AI API, you will implement each stage of the middleware pipeline yourself so that you can understand how information flows through a RAG system.

Your application will simulate an AI-powered course assistant for an online learning platform. Users will ask questions about available courses, and your application will retrieve relevant information from a local knowledge base before generating a simulated AI response.

Step 1 – Create the Project Structure

Create a project folder named **local_rag_assistant** using the following directory structure.

```
local_rag_assistant/  
|  
├─ knowledge/  
|   └─ courses.json  
|  
├─ middleware/  
|   └─ loader.js  
|   └─ chunker.js
```

```
|   ├── retriever.js
|   ├── promptBuilder.js
|   ├── mockLLM.js
|   └── ragPipeline.js
|
|   ├── public/
|   |   ├── index.html
|   |   ├── styles.css
|   |   └── script.js
|   |
|   ├── package.json
|   └── README.md
```

Step 2 – Build a Local Knowledge Base

Inside the **knowledge** folder, create a file named **courses.json**. Populate the file with information about at least **eight** fictional courses offered by an online learning platform. Each course should contain:

- Course title
- Short description
- Difficulty level
- Estimated duration
- Skills covered

The knowledge base will act as the information source for your retrieval pipeline.

Step 3 – Load the Knowledge Base

Create **loader.js**. Write JavaScript that loads the contents of **courses.json** into your application. Your loader should return all available course information so that it can be processed by later stages of the pipeline.

Step 4 – Chunk the Documents

Create **chunker.js**. Rather than treating each course as one large document, divide each course into smaller chunks. For example, one chunk could contain:

- Course title
- Description

Another chunk could contain:

- Difficulty

- Skills
- Duration

Store the generated chunks in a JavaScript array. Include comments explaining why RAG systems divide documents into chunks before retrieval.

Step 5 – Build a Simple Retriever

Create **retriever.js**. Write JavaScript that searches through your chunks and returns the most relevant information for a user's question. Your retriever does **not** need to use machine learning. Instead, create a simple semantic retrieval strategy such as:

- Keyword matching
- Phrase matching
- Weighted keyword scoring

The retriever should return the **three** most relevant chunks for each question. Display these retrieved chunks in the browser so users can see which information was selected.

Step 6 – Build the Prompt

Create **promptBuilder.js**. Construct an augmented prompt that combines:

- A system instruction
- The retrieved document chunks
- The user's original question

Your prompt should resemble the following structure.

```
You are an AI course assistant.  
Use the following information to answer the user's question.  
Context:  
...  
Question:  
...
```

Display the completed prompt on the page. This will allow you to see exactly what information would normally be sent to a large language model.

Step 7 – Simulate the Language Model

Create **mockLLM.js**. Instead of calling an external AI API, create a function that generates a simulated AI response using the retrieved context. Your response should:

- Summarise the retrieved information.
- Refer to the user's question.

- Produce a professional natural-language response.

For example:

```
function generateResponse(question, context) {  
    return `Answer  
Based on the retrieved course information:  
${context}  
This recommendation was produced using the retrieved knowledge base rather  
than a live AI model.  
`;  
}
```

You may improve this function in any way you wish. The goal is to simulate the final stage of a Retrieval-Augmented Generation pipeline without requiring an external AI service.

Step 8 – Orchestrate the RAG Pipeline

Create **ragPipeline.js**. Connect all of the middleware components together. When a user submits a question, your application should execute the following sequence:

1. Load the knowledge base.
2. Chunk the documents.
3. Retrieve the most relevant chunks.
4. Build the augmented prompt.
5. Generate the simulated AI response.
6. Return the response to the interface.

This file should act as the central coordinator of your AI middleware.

Step 9 – Build the User Interface

Create a professional web interface using HTML, CSS, and JavaScript. Your application should include:

- A title and short description.
- A question input field.
- An "Ask Assistant" button.

After a question is submitted, display:

- The user's question.
- The retrieved document chunks.
- The augmented prompt.

- The simulated AI response.

Present each stage inside clearly labelled sections so that users can observe how information moves through the RAG pipeline.

Step 10 – Explain the Pipeline

Below your application, include a section titled **How the Retrieval Pipeline Works**. In your own words, explain:

- How the documents were loaded.
- Why chunking was necessary.
- How retrieval selected relevant information.
- How prompt augmentation improved the request.
- Why a mock language model was used instead of a live AI API.
- How a production RAG application would replace the mock language model with a real one.

Your explanation should demonstrate your understanding of Retrieval-Augmented Generation rather than simply describing your code.



Take note

Ensure that your application runs correctly and demonstrates every stage of the Retrieval-Augmented Generation pipeline.

Important: This practical task assesses your understanding of how Retrieval-Augmented Generation systems work. The focus is on implementing the architecture and middleware pipeline rather than connecting to a commercial AI service. By simulating the language model locally, you can concentrate on understanding the flow of information through a RAG system, which is the foundation of modern AI-powered applications.

Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
